

TradeVault

Audit Database & Data Model

Senior PostgreSQL Architect - Pre-Funding

Juillet 2026

Analyse de la couche base de donnees : schema, RLS, indexes, performance, scalabilite et preparation IA.

Ne pas diffuser en externe

Vue d'Ensemble

TradeVault utilise Supabase (Postgres 15+) avec 20 migrations et 17 tables live (20 definies au total dont 3 non appliquees). Le schema est bien structure: RLS owner-only sur toutes les tables utilisateur, indexes composites couvrant les principaux patterns de requetes, migrations additives (aucune table supprimee). L'architecture montre une bonne maturite avec des patterns comme SECURITY DEFINER pour le rate limiting, des indexes partiels, et un schema extensible.

Cependant, plusieurs problemes structurels empechent le passage en production avec plusieurs milliers d'utilisateurs: migrations dupliquees, colonnes orphelines sans migration, absence de pagination sur les requetes principales, patterns N+1 dans les crons, et divergences entre les types TypeScript et le schema reel.

Tables (17 live, 3 non-appliquees)

- Core: profiles, trades, accounts, missed_opportunities
- Billing: subscriptions, email_log
- Engines: notifications, ai_memory, ai_reports, user_preferences, habits
- Goals: six_month_goals, goal_plans
- Infra: push_subscriptions, ai_rate_limits, processed_webhook_events
- AI OS (non-applique): ai_embeddings, ai_jobs, ai_agent_runs
- Orpheline (sans migration): monthly_reports

1. Schema de Donnees

D01 Critical Migrations dupliquees : profils/trades/storage crees 2-3 fois

P0 | Toutes les migrations 1-6

Probleme: Les migrations 1 et 5 creent toutes deux les tables profils ET trades avec le meme schema. Les migrations 3, 4 et 6 creent les memes 4 politiques de stockage pour le bucket trade-screenshots. Un deployment sur un projet Supabase vierge echouera avec 'relation already exists' sur la migration 5 (et 4, 6 pour les politiques).

Impact: Impossible de provisionner un nouvel environnement (staging, review app, disaster recovery) a partir des migrations. Tout nouveau membre d'equipe doit passer des heures a debugger l'ordre des migrations.

Solution: Supprimer les migrations 2-4 et 6 (redondantes). Conserver la migration 5 comme schema initial unique. Renumeroter les migrations restantes dans l'ordre chronologique.

D02 Critical Colonnes orphelines sans migration : monthly_reports, mae, mfe, trustpilot_*

P0 | Table monthly_reports, colonnes trades.mae/mfe/slippage/is_example, profiles.trustpilot_prompted_at/trustpilot_status

Probleme: 5 colonnes et 1 table sont utilisees par le code applicatif mais n'ont AUCUNE migration pour les creer. monthly_reports est creee dans le type TS (776 lignes) et requetee dans src/app/store/reports.ts. trades.mae/mfe/slippage sont utilises par le moteur d'analyse (engine.ts:111-131). Ces schemas ont probablement ete crees manuellement via le dashboard Supabase et jamais commits.

Impact: Tout deployment sur une base vierge echoue silencieusement ou avec des erreurs SQL. L'analyse engine tourne avec des colonnes manquantes. Detection automatique impossible car le code ne leve pas d'erreur (les colonnes manquantes retournent null en Supabase).

Solution: Ajouter une migration additive pour chaque colonne manquante et la table monthly_reports. Verifier que le code gere les valeurs null pour les anciens trades.

D03 High account_id absent du type Trade applicatif

P1 | src/app/types.ts (type Trade), rowToTrade() dans store/trades.ts

Probleme: Le type Trade (TypeScript) n'inclut pas accountId. La fonction rowToTrade() ignore account_id provenant de la DB. tradeToRow() le reattache via un appel global getActiveAccountId() non type. Impossible de filtrer les trades par compte cote client ou de savoir a quel compte appartient un trade charge.

Impact: Multi-comptes ingerable cote client: si l'utilisateur change de compte, les trades charges n'ont pas de lien vers leur compte d'origine. Le changement de compte force un rechargement complet.

Solution: Ajouter accountId?: string au type Trade. Modifier rowToTrade() pour inclure account_id. Le filtre par compte peut alors se faire cote client sans rechargement.

D04 Medium Aucune contrainte CHECK sur direction, strategy, etc.

P2 | accounts, trades

Probleme: direction accepte n'importe quel texte alors que l'app n'utilise que 'long', 'short', 'be'. strategy accepte tout texte alors que l'app a un ensemble fini de strategies. Aucune validation au niveau DB. Les valeurs aberrantes peuvent s'accumuler sans alerte.

Impact: Integrite des donnees degradee avec le temps. Les analytics et filtres peuvent comporter des valeurs inattendues. Les nouveaux validateurs doivent etre ajoutees en migration, pas en application.

Solution: Ajouter des contraintes CHECK sur direction IN ('long','short','be'), type IN ('personal','prop','demo','live') sur accounts. strategy est un texte libre (utilisateur peut creer ses propres strategies) donc pas de CHECK recommande.

D05 High 3 migrations AI OS non appliquees : ai_embeddings, ai_jobs, ai_agent_runs

P1 | Migration 20260718160000_ai_os_foundation.sql

Probleme: La migration 20 est annotee 'NOT YET APPLIED' (ligne 2). Les 3 tables (ai_embeddings, ai_jobs, ai_agent_runs) n'existent pas dans la base de donnees live. Pourtant, les types TypeScript (rag/types.ts, jobs/types.ts, telemetry.ts) les referencent deja. Les types generes Supabase ne les incluent PAS.

Impact: Si un service background tente d'ecrire dans ai_jobs ou ai_agent_runs, la requete echoue. Les systemes de RAG et de telemetry ne peuvent pas etre deployes sans cette migration. La telemetry IA et le job system sont bloques par cette seule migration non appliquee.

Solution: Appliquer la migration sur la base de production. Regenerer les types Supabase. Alternativement, retirer les references dans le code tant que le systeme n'est pas pret.

2. Supabase RLS & Permissions

S01 Critical Politiques de stockage basees sur le nom de dossier - fuite potentielle

P0 | Bucket storage 'trade-screenshots'

Probleme: Les politiques RLS pour le bucket storage utilisent `(storage.foldername(name))[1] = auth.uid()::text` pour isoler les utilisateurs. Cette approche est moins robuste qu'une veritable politique RLS basee sur les metadonnees. Si le chemin du dossier suit un pattern previsible, un utilisateur pourrait acceder aux screenshots d'un autre utilisateur en devinant son folder name.

Impact: Fuite de donnees potentielle : les screenshots de trades sont des informations sensibles (symboles, strategies, PnL visibles sur l'image). Une fuite pourrait gravement nuire a la confiance et a la reputation.

Solution: Remplacer par des politiques basees sur les metadonnees d'objet (`bucket_id`, `owner`). Utiliser des URLs signees avec expiration au lieu de URLs publiques. Ajouter un champ `owner` dans les metadonnees lors de l'upload.

S02 Medium RLS sur email_log absente - accessible au service_role uniquement

P2 | Table email_log

Probleme: La table email_log n'a AUCUNE politique RLS. Elle estensee etre accessible uniquement via service_role (les migrations laissent les permissions par default). Si un jour la table est accidentellement exposee a authenticated, un utilisateur pourrait lister tous les emails envoyes.

Impact: Fuite de metadonnees (dates d'envoi, types d'email). Risque d'ingenierie sociale si un attaquant connait le cycle de vie des emails.

Solution: Ajouter explicitement: `REVOKE ALL on email_log FROM authenticated, anon. GRANT ALL TO service_role`. Documenter que c'est une table service-role uniquement.

S03 High handle_new_user_billing() sans SET search_path - securite

P1 | Migration 20260717100000_billing.sql, fonction handle_new_user_billing

Probleme: La fonction handle_new_user_billing() est creee sans clause SET search_path. C'est une SECURITY DEFINER (s'exécute avec les droits du proprietaire). Si un attaque manipule search_path, la fonction pourrait executer des fonctions malveillantes definies par un utilisateur dans un schema public.

Impact: Elevation de privileges potentielle. Un utilisateur pourrait inserer des donnees dans des tables auxquelles il n'a normalement pas acces.

Solution: Ajouter SET search_path = public a la fonction handle_new_user_billing(). (La fonction handle_new_user() dans la migration 12 le fait deja correctement.)

S04 Medium profiles.active_account_id FK ON DELETE SET NULL - pas de plan B

P2 | Table profiles, colonne active_account_id

Probleme: Quand le compte actif est supprime, active_account_id passe a NULL via ON DELETE SET NULL. Le code applicatif ne gere pas ce cas: aucun fallback vers un autre compte, pas de redirection. L'utilisateur voit un ecran vide.

Impact: L'utilisateur peut se retrouver bloque avec un ecran vide ou des trades invisibles apres avoir supprime son compte actif. Mauvais pattern UX + DB.

Solution: Modifier la contrainte pour ON DELETE SET DEFAULT (vers le compte par default). Ajouter un trigger BEFORE DELETE qui selectionne un autre compte par default avant de supprimer.

3. Performance Database

P01 Critical loadUserTrades: SELECT * sans pagination ni limite

P0 | src/app/store/trades.ts:90-93

Probleme: loadUserTrades() execute supabase.from('trades').select('*').eq('user_id', userId) sans .limit(), sans .range(), sans colonnes explicites. A 5000 trades par utilisateur, c'est ~2.5-10MB transferees a chaque chargement de page. Le mapper rowToTrade() n'utilise que 22 colonnes mais le SELECT * ramene TOUTES les colonnes dont notes (texte long), screenshots (tableau de chemins), confluences (tableau).

Impact: Temps de chargement qui se degrade lineairement avec l'historique. Sur mobile, 10MB+ par chargement est prohibitif. A 100 utilisateurs simultanés, le serveur Supabase transfert potentiellement 1GB+ pour les seules requetes trades.

Solution: 1) Remplacer SELECT * par SELECT avec les colonnes exactes de rowToTrade (22 colonnes). 2) Ajouter une pagination: les 200 trades les plus recents, avec 'Load more'. 3) Ajouter un index couvrant pour les colonnes selectionnees. 4) Fixer loadMissedOpportunities() avec le meme pattern.

P02 Critical N+1 dans handleMonthlyReportsCron - 2000+ requetes pour 1000 users

P0 | src/backend/monthly-reports.server.ts:302-312

Probleme: Le cron mensuel boucle sur chaque userId et execute: generateReportForUser() (lecture trades + profiles) puis notifyUser() (lecture push_subscriptions). Pour 1000 utilisateurs, c'est ~3000 requetes sequentielles. Pas de parallelisation, pas de batch prefetch.

Impact: Le cron prend des heures a s'executer. Sur Vercel (max 60s/executation serverless), le cron timeout avant d'avoir traite tous les utilisateurs. Les rapports sont partiellement generes sans alerte.

Solution: 1) Batch-fetch profiles + push_subscriptions AVANT la boucle avec .in('id', userIds). 2) Paralleliser les generateReportForUser avec Promise.allSettled (limite de 10 concurrents). 3) Ajouter un compteur avec timeout pour tracker la progression.

P03 High N+1 dans handleGoalRemindersCron et handleLifecycleCron

P1 | src/backend/goal-reminders.server.ts:101-138, src/backend/lifecycle-emails.server.ts:123-158

Probleme: Le cron des rappels d'objectifs et le cron des emails de cycle de vie utilisent le meme pattern: boucle for sur les utilisateurs + 2 requetes individuelles par iteration (push_subscriptions + profiles). Meme probleme que P02 a plus petite echelle. Le cron des emails de cycle de vie est QUOTIDIEN donc l'impact s'accumule.

Impact: Coûts Supabase eleves (chaque requete est facturable). Timeouts possibles sur des jeux de donnees moyens. Consommation excessive d'appels Supabase.

Solution: Batch prefetch par .in() avant la boucle. Parallelisation intra-boucle. Passer a une seule requete par type de donnees au lieu de N.

P04 High SELECT * sur missed_opportunities et notifications

P1 | src/app/store/missed.ts:52, src/modules/notifications/store.ts:29

Probleme: loadMissedOpportunities() fait SELECT * sans pagination. loadNotifications() a un .limit(50) mais fait aussi SELECT * alors que seuls 8 champs sont utilises.

Impact: Gaspillage de bande passante et memoire. missed_opportunities peut contenir des colonnes texte longues (what_happened, lesson_learned). notifications a un LIMIT 50 donc l'impact est moindre mais le pattern est identique.

Solution: Colonnes explicites partout. Remplacer SELECT * par les colonnes exactes utilisees par les mappers respectifs.

P05 Medium loadMonthlyReports sans pagination

P2 | src/app/store/reports.ts:12-17

Probleme: loadMonthlyReports() recupere TOUS les rapports mensuels sans limite. Bien que le nombre de rapports soit faible (1/mois, ~12/an), le contenu (report est jsonb avec graphiques et statistiques) peut etre volumineux.

Impact: Apres 5 ans d'utilisation, 60 rapports × ~50KB = 3MB charges a chaque acces a la page Reports.

Solution: Ajouter .limit(12) (12 derniers mois). Ajouter un bouton 'Charger plus' ou une pagination pour les rapports plus anciens.

P06 Low Pas d'index sur profiles.active_account_id

P3 | Table profiles

Probleme: La colonne active_account_id n'a pas d'index. Bien que chaque utilisateur n'ait qu'une seule ligne, les requetes qui filtrent par active_account_id (ex: 'trouver le user lie a ce compte') necessitent un seq scan.

Impact: Impact negligeable pour les petits volumes mais pourrait devenir un probleme avec des centaines de milliers d'utilisateurs et des requetes administratives.

Solution: Ajouter CREATE INDEX IF NOT EXISTS profiles_active_account_idx ON public.profiles (active_account_id) WHERE active_account_id IS NOT NULL.

4. Analyse de Scalabilite

Estimation par palier d'utilisateurs

Hypotheses: 5000 trades/utilisateur, 100 colonnes/profils, 50 notifications/utilisateur, 50 evenements memoire/utilisateur. Row width estime: trades ~800 bytes, missed ~600 bytes, profiles ~2KB.

Dimension	1 000 users	10 000 users	100 000 users
Total trades	5M	50M	500M
Total missed	500K	5M	50M
Total notifications	50K	500K	5M
Taille trades (est.)	4 GB	40 GB	400 GB
Taille missed (est.)	300 MB	3 GB	30 GB
Taille notifications	50 MB	500 MB	5 GB
Chargement page (select *)	4 MB	4 MB	4 MB*
Cron mensuel (req.)	~3000 req	~30K req	~300K req
Requetes/jour (est.)	~100K	~1M	~10M

* select(*) non paginee : 5000 trades charges par utilisateur independamment du nombre d'utilisateurs. Le Goulot d'etranglement est le nombre de trades PAR utilisateur, pas le nombre d'utilisateurs. Chaque utilisateur charge ses 5000 trades a chaque visite. A 100K utilisateurs, le probleme n'est pas la taille des donnees mais le nombre de connexions simultanees.

Limites identifiees

- LoadUserTrades (P0) : sans pagination, chaque utilisateur charge tout son historique a chaque visite. Irréaliste a partir de 500 trades.
- Crons N+1 (P0) : les crons mensuels/quotidiens ne passeront pas l'échelle sans prefetch par lot. Sur Vercel serverless, timeout a 60s.
- SELECT * (P1) : le cout reseau est multiplie par 2-3x vs colonnes explicites.
- Indexation : les indexes composites existants (trades_user_account_date_idx) sont bien concus et tiendront jusqu'a 100M de lignes.
- RLS : les politiques owner-only sont efficaces car Postgres utilise l'index user_id pour filtrer. Pas de degradation RLS attendue.
- ai_memory : 40 entrees par utilisateur, chaque entree jusqu'a 2000 chars. A 100K utilisateurs, 4M lignes. L'index composite couvre.
- jsonb (trading_rules, trading_plan, prefs) : pas d'index GIN. Les requetes sur les champs jsonb feront du seq scan. A surveiller.

5. Preparation IA

Etat actuel

La couche DB pour l'IA est deja bien preparee : ai_memory pour la memoire episodique, ai_reports pour les rapports generes, notifications pour les alertes. Les indexes composites (user_id, kind, created_at) couvrent les patterns de requete. RLS owner-only partout.

Points bloquants pour l'architecture IA future

AI01	Critical	Migration AI OS non appliquee : RAG, jobs, telemetrie bloques
P0 Migration 20260718160000 (ai_os_foundation)		
Probleme: Les 3 tables AI OS (ai_embeddings, ai_jobs, ai_agent_runs) sont definies dans une migration marquee 'NOT YET APPLIED'. Sans elles : pas de memoire semantique (RAG), pas de file d'attente de jobs background, pas de telemetrie des agents. Le AI Operating System ne peut pas fonctionner.		
Impact: Tout le developpement IA au-dela du chat reactif est bloque. Les agents, le routage, la personnalisation avancee dependent de ces tables.		
Solution: Appliquer la migration sur la base live. Generer un plan de migration pour les environnements existants.		
AI02	Medium	ai_memory sans index GIN/trgm pour recherche plein texte
P2 Table ai_memory, colonne content		
Probleme: ai_memory.content est un texte libre recherche par le coach IA. Actuellement, l'index (user_id, kind, created_at) couvre le tri mais pas la recherche de contenu. Si le coach veut retrouver un fait specifique ('quand a-t-il depasse sa limite de risque ?'), la recherche est sequentielle.		
Impact: Les performances de recherche memoire se degradent avec le volume. Le coach IA ne peut pas retrouver rapidement des faits anciens.		
Solution: Ajouter un index GIN avec pg_trgm pour la recherche full-text sur ai_memory.content. CREATE EXTENSION pg_trgm; CREATE INDEX ai_memory_content_trgm_idx ON ai_memory USING gin (content gin_trgm_ops);		
AI03	Low	Pas de champ user_id dans les types d'evenements de l'Event Bus
P3 DomainEvents interface (events/types.ts)		
Probleme: Les payloads d'evenements incluent deja userId. La DB n'a pas encore de table d'evenements persistes (event sourcing). Pour le futur: la telemetrie IA aurait besoin de tracer les evenements declencheurs.		
Impact: Pas d'impact immediat. A surveiller quand la proactivite IA sera deployee.		
Solution: Ajouter une table domain_events pour l'event sourcing quand le besoin se presentera. Pas urgent.		

6. Recommandations Prioritaires

Les 10 actions DB avant production

P0 Nettoyer les migrations dupliquees

Supprimer les migrations 2-4, 6. Migration 5 = schema initial unique. Ajouter les colonnes orphelines (mae, mfe, slippage, is_example, trustpilot_*, monthly_reports) dans une nouvelle migration. Renumeroter.

Estimation: 1 jour

P0 Pagination + colonnes explicites sur loadUserTrades

Remplacer SELECT * par 22 colonnes exactes. Ajouter LIMIT 200 + range. Pattern 'load more' ou infinite scroll. Meme fix pour missed_opportunities.

Estimation: 2 jours

P0 Batch prefetch dans handleMonthlyReportsCron

Remplacer les N requetes individuelles par 2-3 requetes batch avec IN (userIds). Paralleliser generateReportForUser (Promise.allSettled, limite 10). Ajouter timeout monitoring.

Estimation: 1-2 jours

P0 Appliquer la migration AI OS + regenerer les types

Appliquer 20260718160000 sur la base live. Regenerer les types Supabase pour inclure ai_embeddings, ai_jobs, ai_agent_runs.

Estimation: 0.5 jour

P0 Securiser les politiques de stockage

Remplacer le folder-name check par une verification basee sur les metadonnees d'objet. Utiliser des URLs signees avec expiration.

Estimation: 1-2 jours

P1 Fixer les N+1 dans handleLifecycleCron + handleGoalRemindersCron

Batch prefetch avant la boucle. Parallelisation intra-boucle.

Estimation: 1 jour

P1 Remplacer tous les SELECT * par des colonnes explicites

Notifications (store.ts), tout SELECT * dans le codebase. Verifier avec un lint rule ou un grep.

Estimation: 0.5 jour

P1 Securiser handle_new_user_billing() avec SET search_path

Ajouter SET search_path = public. Deja fait dans handle_new_user(), mais pas dans la fonction billing.

Estimation: 5 min

P2 Ajouter les contraintes CHECK manquantes

direction IN ('long','short','be'), type IN ('personal','prop','demo','live'). Auditer les colonnes texte qui devraient etre contraintes.

Estimation: 0.5 jour

P2 Ajouter accountId au type Trade applicatif

Modifier src/app/types.ts, rowToTrade, tradeToRow. Permettre le filtrage multi-compte cote client.

Estimation: 1 jour

Score Global et Tableau Recapitulatif

Schema & Modelisation: 55/100 (55%)

RLS & Securite: 60/100 (60%)

Performance & Indexes: 35/100 (35%)

Scalabilite 1K-100K users: 30/100 (30%)

Preparation IA: 40/100 (40%)

Migrations & Integrite: 25/100 (25%)

Type Safety (TS vs DB): 45/100 (45%)

SCORE GLOBAL DATABASE: 290/700 (41%)

Tableau Recapitulatif

Nombre total de problem	19
Critical (P0)	8
High (P1)	5
Medium (P2)	4
Low (P3)	2
Temps d'implementation	~10-15 jours/homme
Tables live	17
Tables sans migration	1 (monthly_reports)
Colonnes orphelines	6
Migrations dupliquees	4 fichiers
SELECT * non pagines	3 queries
Patterns N+1	3 crons

Resume Executif

La couche base de donnees de TradeVault est architecturalement saine : RLS owner-only sur 16/17 tables, indexes composites bien concus, schema extensible avec migrations additives, separation claire entre donnees utilisateur et infrastructure. Les fondations sont solides pour un SaaS en pre-funding.

CE QUI EST EXCELLENT :

- RLS owner-only appliquee partout, aucune policy trop permissive
- Indexes composites couvrant les patterns de requete reels (user_id + date + account)
- Migrations additives uniquement (DROP TABLE jamais utilise)
- Pattern SECURITY DEFINER pour le rate limiting atomique (consume_ai_quota)
- Separation des triggers signup bien isolee (handle_new_user + handle_new_user_billing)
- Schema IA prepare (ai_memory, ai_reports, notifications) avec indexes
- Aucune raw SQL dans le code applicatif

CE QUI DOIT ETRE CORRIGE AVANT PRODUCTION :

- (P0) Migrations dupliquees : impossible de provisionner un environnement vierge. Priorite #1 absolue.
- (P0) 6 colonnes + 1 table sans migration : la base live et le code divergent du schema migration. Risque d'echec au deployment.
- (P0) Pagination absente sur trades et missed_opportunities : chaque utilisateur charge TOUT son historique a chaque visite. Temps de chargement lineaire avec le nombre de trades.
- (P0) N+1 dans les 3 crons : les taches planifiees ne passeront pas l'echelle. Le cron mensuel timeout sur Vercel (60s).
- (P0) Migration AI OS non appliquee : la telemetrie, le RAG et les jobs background sont bloques par une seule migration non executee.
- (P0) Politiques de stockage basees sur le folder name : risque de fuite de donnees entre utilisateurs.

VERDICT : La base de donnees peut tenir 1000 utilisateurs aujourd'hui avec les correctifs P0 ci-dessus. Pour 10 000 utilisateurs, la pagination et la resolution des N+1 sont indispensables. Pour 100 000 utilisateurs, l'architecture actuelle tiendra sous reserve d'ajouter : connection pooling (PgBouncer), read replicas pour les analytics, et un cache Redis pour les trades frequemment accedes.